Code Iterations

```
#instances of stuff
sensor = Orientation_Sensor()
red_led = LED(6)
yellow_led = LED(16)
green_led = LED(1)
servo = Servo(8)


SAMPLE_TIME = 0.1
window = 10
PERSIST_DURATION = 5 # seconds

#thresh (tune these!!!)
GYRO_LOW = 2
GYRO_MED = 10

ACCEL_LOW = 0.7
ACCEL_MED = 1.5

#numpy buffers or 2D array of zeroes, 10 elements total each element with [x, y]
gyro_buffer = np.zeros((window, 3))
accel_buffer = np.zeros((window, 2))
```

Feb 3rd, 2026: Initially figuring out GPIO stuff and working with numpy to get arrays of zeroes.

```
def update_buffer(buffer, new_val):
    #basically to make a rolling window
    #FIRST STEP IS TO SHIFT
    #slicing the list from the second element
    #and assigning them to all elements from the beginning up to the last element
    buffer[:-1] = buffer[1:]
    #now inserting the new last element
    buffer[-1] = new_val
```

```
while True:
    # get values
    gyro = np.array(sensor.gyroscope())
    # slicing to ignore Z-axis (gravity) bc it's always 9.8 when testing values
    accel = np.array(sensor.accelerometer()[:2])

    # update rolling arrays
    update_buffer(gyro_buffer, gyro)
    update_buffer(accel_buffer, accel)

    #the axis parameter tells the function which dimension of the array to collapse, here it goes down the
    avg_gyro = np.mean(gyro_buffer, axis=0)
    avg_accel = np.mean(accel_buffer, axis=0)

    gyro_mag = magnitude(avg_gyro)
    accel_mag = magnitude(avg_accel)
```

Feb 5th, 2026: Worked on a way to shift the buffer and accommodate new values, rolling average is incorporated into the main function.

```
# determine the next state based on current sensor readings
if gyro_mag < GYRO_LOW and accel_mag < ACCEL_LOW:
    new_potential = "GREEN"
elif gyro_mag < GYRO_MED and accel_mag < ACCEL_MED:
    new_potential = "YELLOW"
else:
    new_potential = "RED"

# Logic for persistence timer
if new_potential != current_state:
    # If this is a new state, start timing it
    if new_potential != potential_state:
        potential_state = new_potential
        state_start_time = time.time()

    # Check if the potential state has been held for enough time
    elapsed = time.time() - state_start_time
    if elapsed >= PERSIST_DURATION:
        current_state = potential_state
        set_state(current_state)
        print(f"--- STATE CHANGED TO {current_state} ---")
else:
    # We are in the current state, reset potential tracking
    potential_state = current_state
```

Feb 7th: Worked on persistence logic and state change using elapsed time.

```python
def calculate_probability(mean_val, std_val, threshold):
    """Calculates the probability that the actual movement exceeds the threshold.
    Uses Z-score and Normal distribution CDF."""
    if std_val < 0.001:  # avoid division by zero for steady readings
        #the logic here is simple.
        #if the standard deviation is too small, it means the sensor is not moving much, or the
        #so the each data point relatively corresponds to the mean value, and if the mean value
        #if the mean value is below the threshold, it means the sensor is not moving enough
        return 100.0 if mean_val > threshold else 0.0

    # Z-score: how many standard deviations the threshold is from the mean
    # We want the probability that true reading > threshold
    z = (mean_val - threshold) / std_val

    # normal CDF using math.erf
    prob = 0.5 * (1 + math.erf(z / math.sqrt(2)))
    return prob * 100
```

Feb 8th: Worked on standard deviation and probability logic assuming a normal distribution of movement during nighttime to qualify sleepwalking.

Haaziq Iq Add new sleepwalking detection code

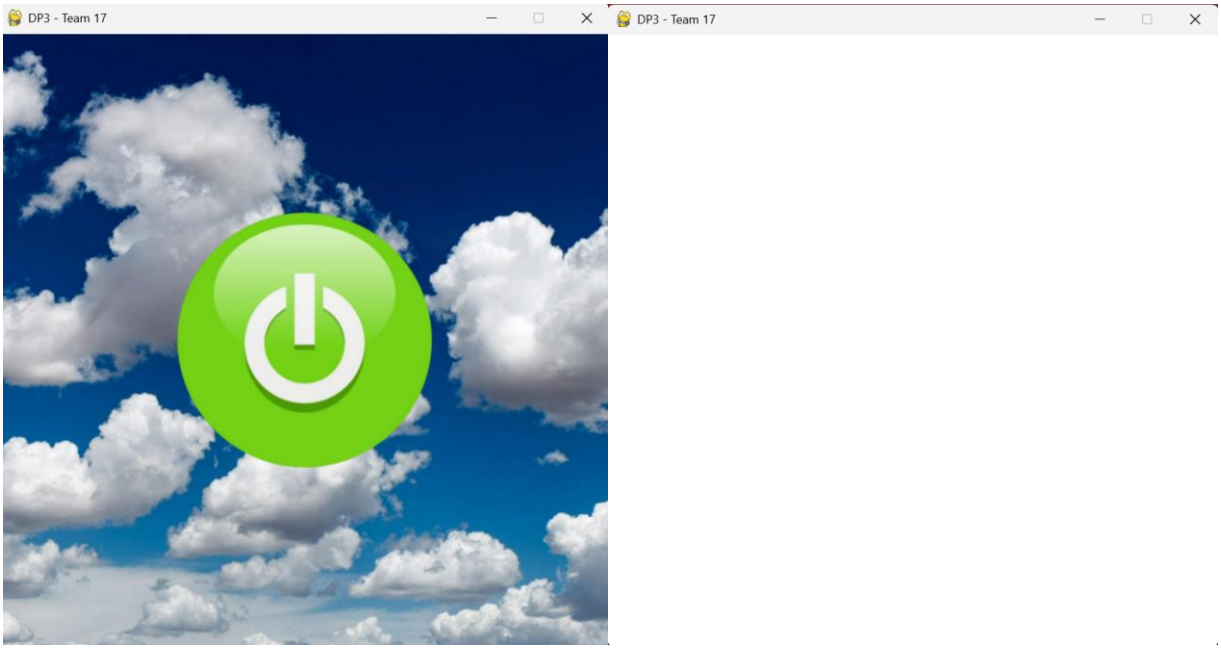
Code Blame 184 lines (147 loc) · 6.72 KB Raw

```python

#imports

import numpy as np
from sensor_library import *
import time
import math
from gpiozero import LED, Servo
#(to quit)
import sys

#instances of stuff
sensor = Orientation_Sensor()
red_led = LED(6)
yellow_led = LED(16)
green_led = LED(1)
servo = Servo(8)
```

Feb 13th: Flushed out full code working with values, HOWEVER. displayless, pushed onto GitHub.

pygametesting.py

```python
import pygame
import sys


pygame.init()
pygame.mixer.init()


screenwidth = 600
screenheight = 600
window = pygame.display.set_mode((screenwidth, screenheight))
pygame.display.set_caption("DP3 - Team 17")


#Loading background and scaling to fit the screen
background = pygame.image.load("sky600.jpg")
background = pygame.transform.scale(background, (screenwidth, screenheight))

# Loading power button image
button_on = pygame.image.load("poweron250.png")
# centering the button in the middle of the screen
button_rect = button_on.get_rect(center=(screenwidth // 2, screenheight // 2))

# app states
homepage = "home"
blankpage = "blank"
current_state = homepage

#main
running = True
while running:
```



Feb 15th: Ran pygame test code on VScode, working with different different images and flicking between two app states a blank page and a cloud page. Worked with music shoutout giveon.

Feb 15th – 19th: Attempted to merge the sensor logic with the display 7 iterations of this. All on GitHub.

```
# checking if any value of the buffer are exactly all zeroes, the buffer is still filli
is_all_zero = 0
for i in range(len(buffer)):
    is_all_zero = 0
    for val in buffer[i]:
        if val == 0:
            is_all_zero += 1
    if is_all_zero == len(buffer[i]):
        return buffer, None, None
```

```
if np.all(gyro == 0) or np.all(accel == 0):
    print("Sensor Error. Reading all zeroes, skipping this reading.")
    time.sleep(SAMPLE_TIME)
    continue
```

Feb 22nd: Once merging was complete, worked on adhering to the project module. Exception handling for when the sensor is filling up and for when there is an error in sensor readings. For the position sensor (readings have very minimal chance to all be 0, because there must be at least some acceleration always on the object, and some small gyro angles read.

GitHub repo below, with most changes made being reflected.

https://github.com/haaziq11/nomoresleepwalking-/tree/master